ECS 160 – Mock Testing

Instructor: Tapti Palit

Teaching Assistant: **Gabe Bai**

Agenda

- Mock testing

Why Testing Matters

- Software behavior depends on external components
  - Databases
  - Network services
  - File systems
- Real dependencies are:
  - Slow
  - Unreliable
  - Hard to control
- Goal: test logic in isolation

Problem with Traditional Unit Tests

- Example issues:
  - Requires database setup
  - Requires network availability
  - Test failures unrelated to logic
  - Hard to reproduce edge cases
- Unit tests should fail only when the unit is wrong.

What Is Mock Testing?

- Mock testing:
  - Replaces real dependencies with fake objects
  - These objects:
    - Return predefined values
    - Record interactions
- Allows:
  - Isolation
  - Deterministic tests
  - Faster execution

Mock vs Real Object

Aspect	Real Object	Mock Object
External state	Yes	No
Speed	Slow	Fast
Control behavior	Limited	Full
Deterministic	No	Yes

Types of Test Doubles

- Dummy: placeholder, not used
- Stub: returns fixed values
- Mock: verifies interactions
- Spy: partial real object + tracking

Common Java Mocking Tools

- Mockito (most common)
- EasyMock
- JMock

Example Scenario

```
class UserService {
    private UserRepository repo;

    public UserService(UserRepository repo) {
        this.repo = repo;
    }

    public String getUserName(int id) {
        return repo.findById(id).getName();
    }
}
```

Without Mocking: Problems

- Issues:
  - Need database
  - Test setup complicated
  - Slow execution
  - Hard to test error paths

Introducing Mockito

- Core idea:
  - Replace UserRepository with a mock
  - Define expected behavior
- Key concepts:
  - mock()
  - when()
  - thenReturn()

Creating a Mock

```
UserRepository repo = mock(UserRepository.class);
UserService service = new UserService(repo);
```

- No real database
- Object exists only in memory

Stubbing Behavior

- Whenever findById(1) is called
- Return user

```
User user = new User("Alice");
when(repo.findById(1)).thenReturn(user);
```

Writing the Unit Test

```
@Test
void testGetUserName() {
    UserRepository repo = mock(UserRepository.class);
    UserService service = new UserService(repo);

    when(repo.findById(1)).thenReturn(new User("Alice"));

    assertEquals("Alice", service.getUserName(1));
}
```

Verifying Interactions

```
verify(repo).findById(1);
```

- Ensures correct collaboration
- Detects missing or extra calls

Verifying Call Counts

- Use cases:
  - Performance-sensitive logic
  - Avoid unintended side effects

```
verify(repo, times(1)).findById(1);
verify(repo, never()).delete(any());
```

Mocking Exceptions

- Allows testing:
  - Error handling
  - Fallback logic
  - Retry mechanisms

When NOT to Use Mocks

- Avoid mocking:
  - Simple data objects
  - Value classes
  - Core language or library logic
- Rule of thumb:
  - Mock collaborators, not data

Common Mocking Pitfalls

- Over-mocking (tests mirror implementation)
- Mocking internal methods
- Brittle tests due to interaction overuse
- Guidelines:
  - Prefer state verification over interaction verification

Best Practices Summary

- Keep tests small and focused
- Mock external dependencies only
- One behavior per test
- Tests should read like documentation